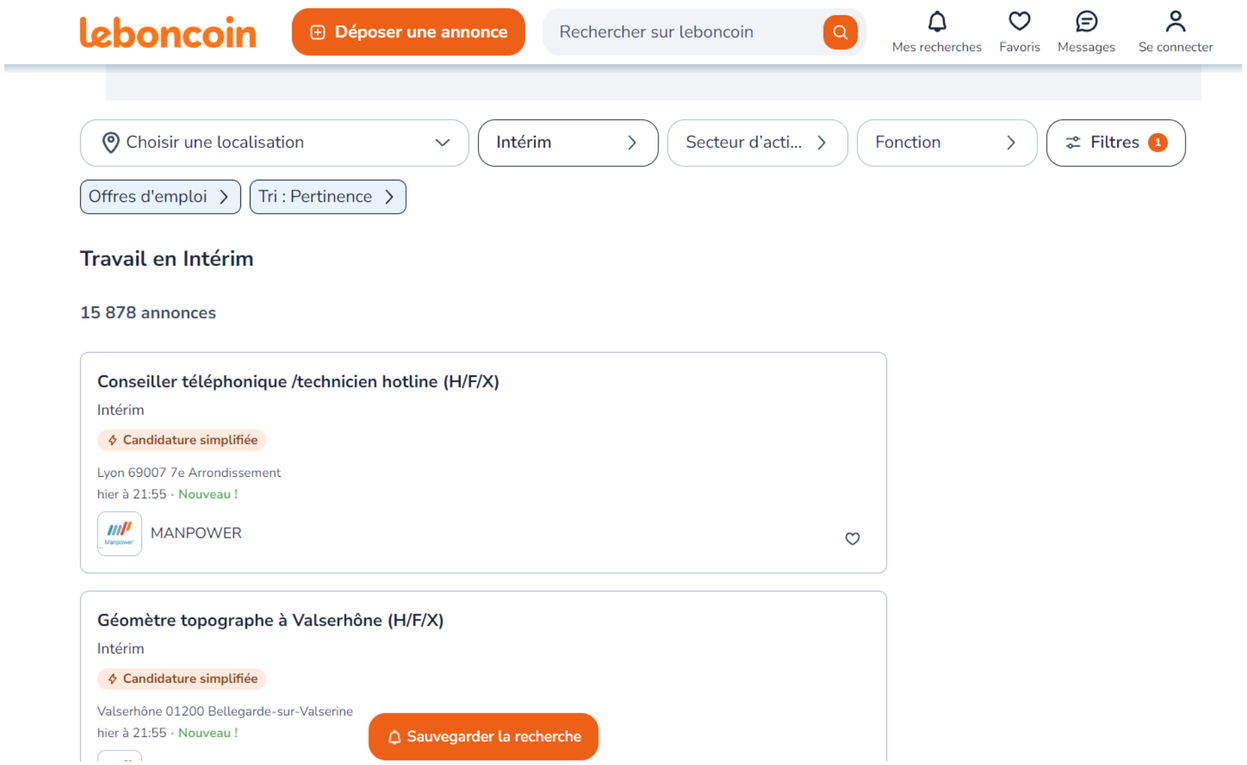
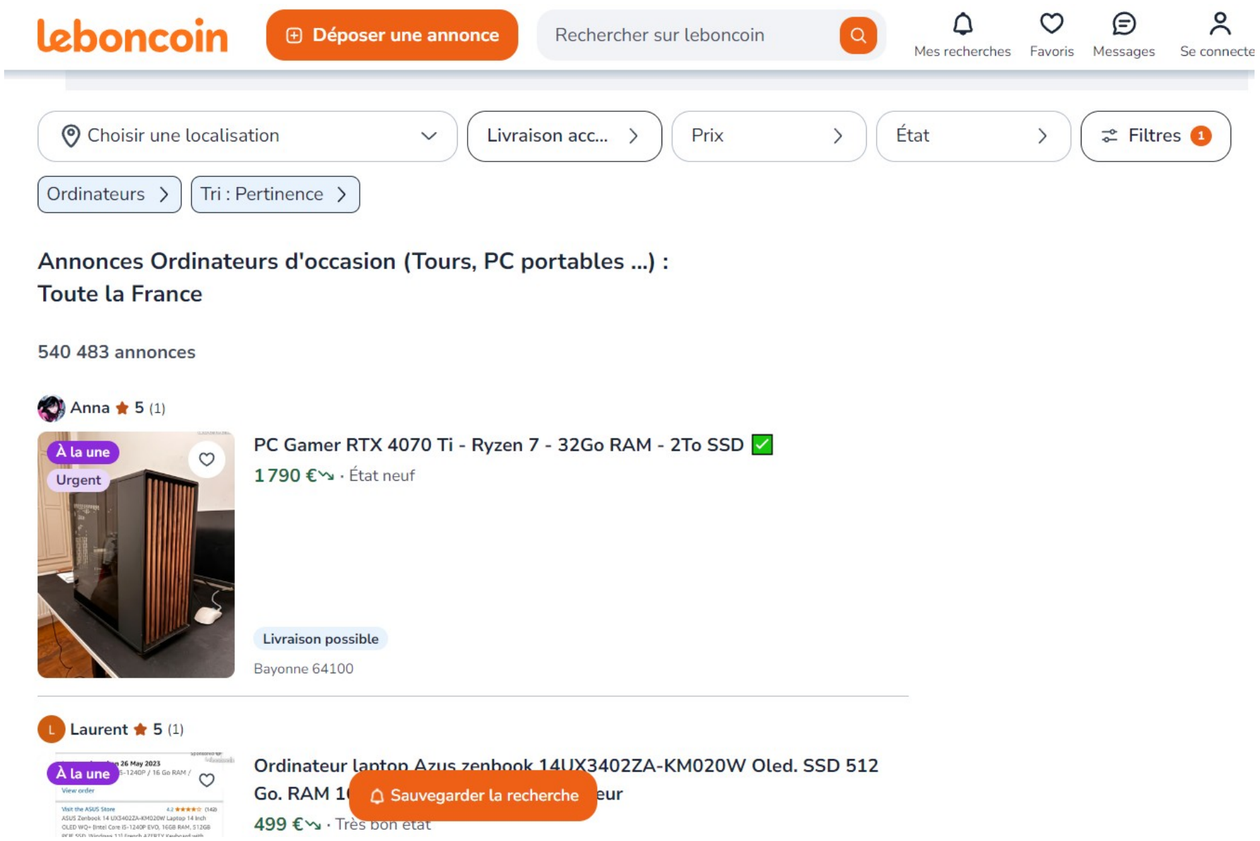
Abstraction et polymorphisme

Dany Siriphol – CODA 1 ère année - 2025

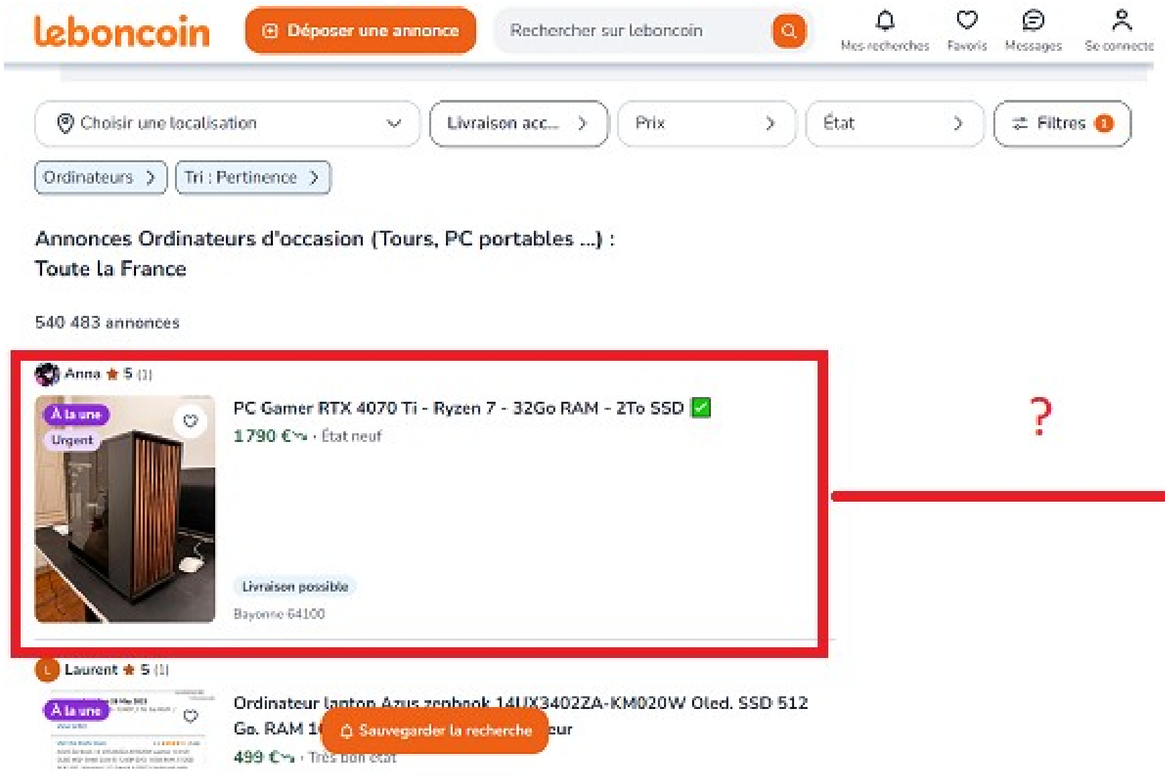
Comment avoir un code modulaire?





Comment avoir un code modulaire?

Comment gérer tout les cas ? (sans tout copier coller)



?

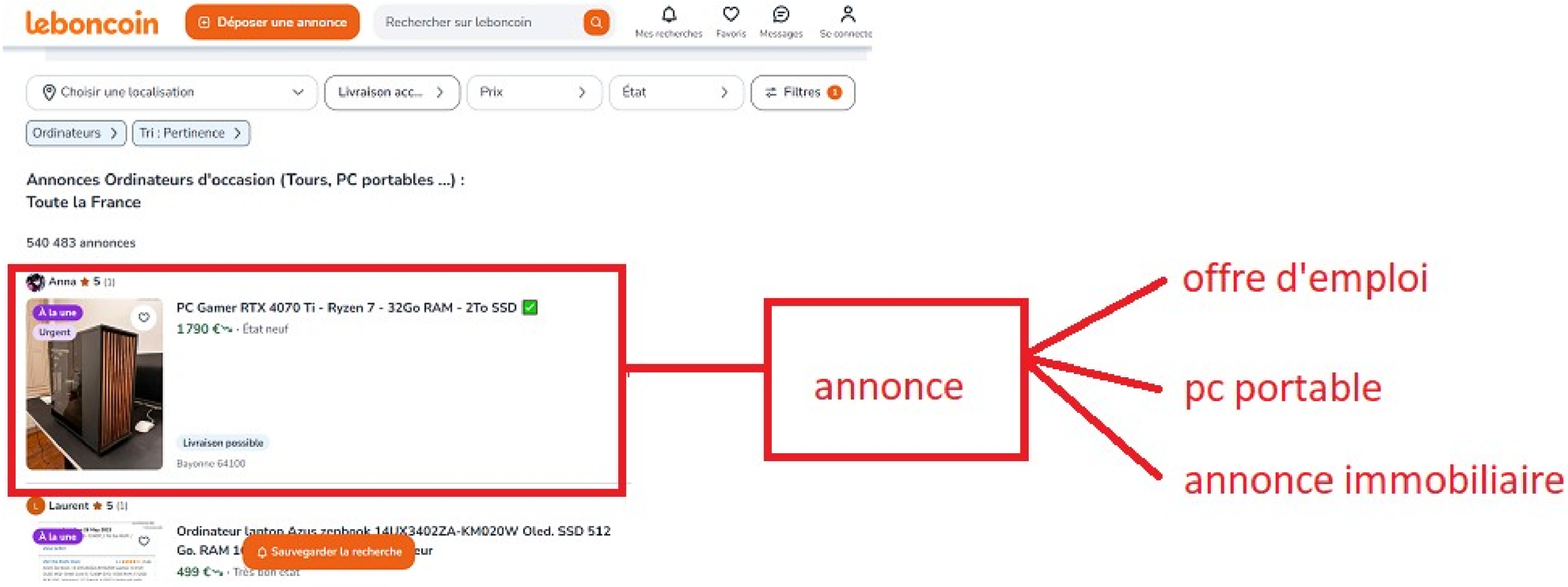
offre d'emploi

pc portable

annonce immobiliaire

Abstraction

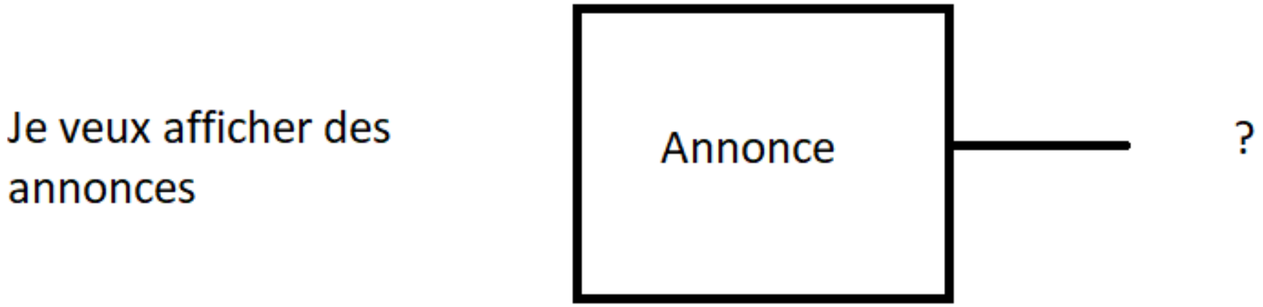
Faire abstraction des détails et considérer tout comme une annonce



Abstraction

En psychologie, la pensée abstraite, dite aussi capacité d'abstraction, désigne la capacité de l'esprit à créer et utiliser des concepts dans le raisonnement. (Wikipedia)

Conception retirée de son contexte en vue de pratiquer un raisonnement.



Interfaces

Interface

Syntaxe

Une interface décrit le comportement qui doit être implémenté par une classe.

Elle est composé d’un ensemble de méthodes et de constantes
Une interface n’est pas une classe et ne peut pas être instancié
Seules les signatures des méthodes sont définies.

```
public interface IDatabaseConnection {
    public void connect(String login, String password);
    public void disconnect();
}
```

Interface

Les classes peuvent « hériter » d'une interface via le mot clé **implements**

```
public interface IDatabaseConnection {
    public void connect(String login, String password);
    public void disconnect();
}
```

```
public class MySQLConnection implements IDatabaseConnection {
```

Interface

Une classe qui **impléments** une interface à l'obligation de définir toutes les méthodes de l'interface

On peut considérer une interface comme un « **contrat** » que doit remplir une classe

```
public class MySQLConnection implements IDatabaseConnection {

    @Override
    public void connect(String login, String password) {
        // TODO Auto-generated method stub

    }

    @Override
    public void disconnect() {
        // TODO Auto-generated method stub

    }

}
```

Interface

Les interfaces peuvent également contenir des propriétés statiques publiques
Ces constantes ne peuvent être que publiques et sont accessibles dans tout le programme

```
public interface IPokemon {
    public static int TYPE_FIRE = 1;
    public static int TYPE_WATER = 2;

    public void attack(IPokemon pokemon);
}
```

```
if( i == IPokemon.TYPE_FIRE ) {

}
```

Interface et typage

> Comme une classe, une interface définit un nouveau type utilisable dans le programme
>
> Une interface est définie comme un type abstrait utilisé pour spécifier le comportement d'une classe

```
IDatabaseConnection connection = new MySQLConnection();
```

Interface et typage

Une interface peut être implémentée par plusieurs classes

Si une classe implémente l'interface A alors cette classe est de type A

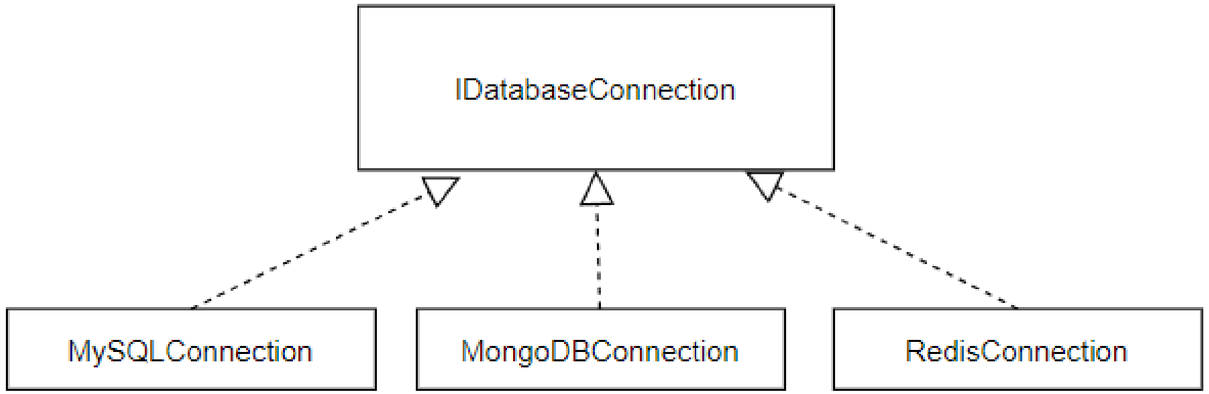
```
public class MongoDBConnection implements IDatabaseConnection {

public class MySQLConnection implements IDatabaseConnection {
```

```
IDatabaseConnection connection = new MySQLConnection();

IDatabaseConnection connection = new MongoDBConnection();
```

Interface et typage

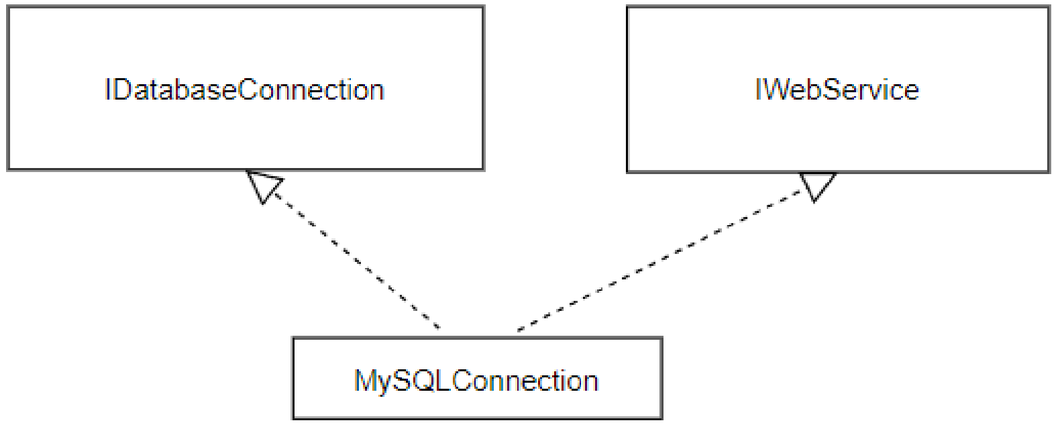
Une interface peut donc avoir plusieurs implémentations

Cela permet d’avoir une famille de classe partageant un même comportement



Interface et typage

Une classe peut implémenter plusieurs interfaces contrairement à l'heritage




```
public class MySQLConnection implements IDatabaseConnection, IWebService {

}
```

Interface et héritage

Syntaxe

Une classe peut hériter d’une classe et implémenter des interfaces en meme temps

```
public class Employee extends Person implements IWorker{
```

Classes abstraites

Classe abstraite

Syntaxe

Une classe est une classe qui possède au moins une **méthode abstraite**

Une méthode abstraite est une méthode qui ne possède pas d'implémentation de façon similaire à une interface, c'est une implémentation partielle d'une fonctionnalité

```
public abstract class ChessPiece {
    protected int x;
    protected int y;

    public ChessPiece(int x, int y) {
        this.x = x;
        this.y = y;
    }

    public abstract boolean checkMove(int x, int y);
}
```

Classe abstraite

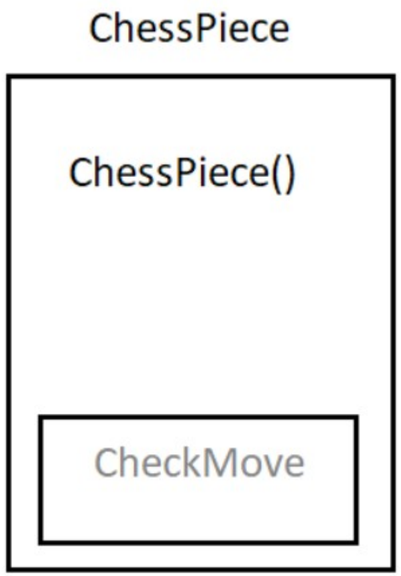
Une classe abstraite ne peut pas être instanciée car elle possède une méthode abstraite et sans implémentation

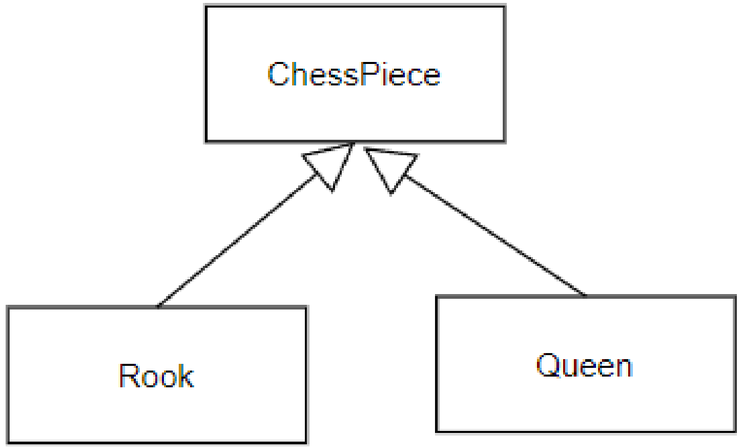
```
ChessPiece p = new ChessPiece(1,1);
```

Classe abstraite

Une classe abstraite sert de modèle pour des classes « concrétés ». Son but et d’être hérite par une autre classe qui va définir les méthodes abstraites

Une classe abstraite est comme une classe à « trou » qu’une autre classe doit compléter





Héritage d’une classe abstraite

```java
public class Rook extends ChessPiece{

    public Rook(int x, int y) {
        super(x, y);
    }

    @Override
    public boolean checkMove(int x, int y) {
        return this.x == x || this.y == y;
    }
}
```

Classe abstraite et typage

Syntaxe

Une classe abstraite devient un type disponible dans le programme. Il est possible d'utiliser une des classes enfants comme classe concrète

```
ChessPiece p = new Rook(1,1);
p.checkMove(2, 1);
```

```
ChessPiece p = new Queen(1,1);
p.checkMove(2, 1);
```

Polymorphisme

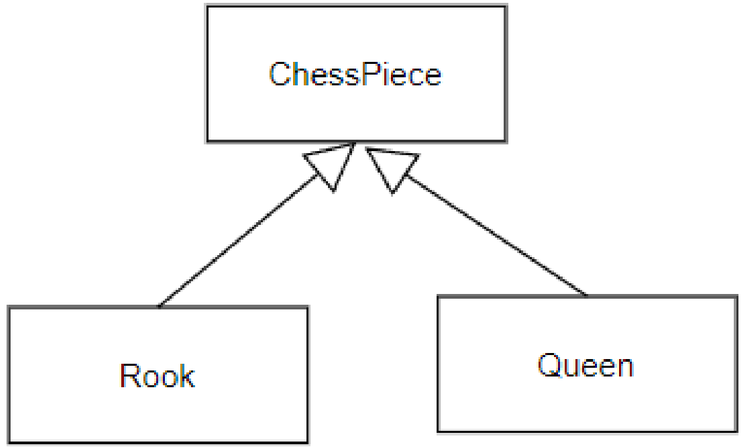
Polymorphisme

Définition

Polymorphisme vient du grec et signifie qui peut prendre plusieurs formes.

Le polymorphisme est le fait de baser son programme sur des types abstraits et non des classes concrètes. Cela consiste à fournir une interface commune à des classes de différents types

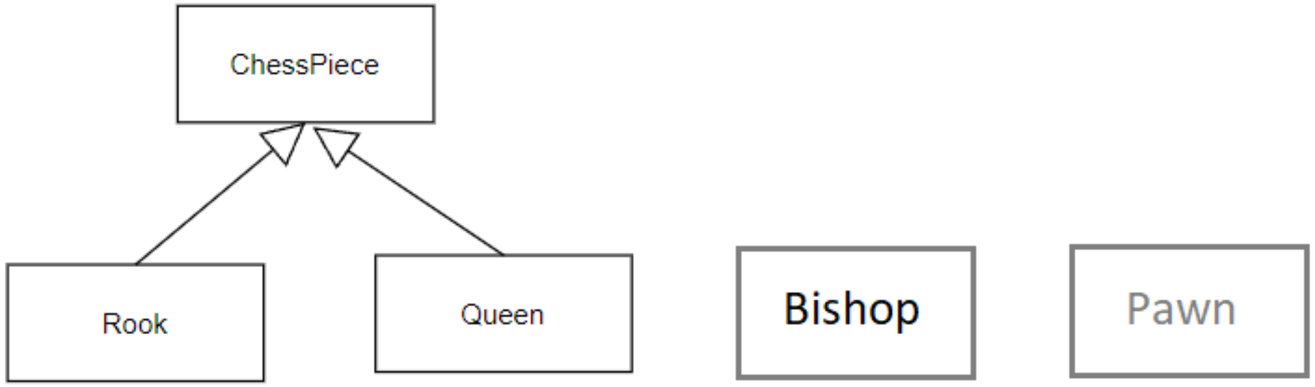
```
ChessPiece p = ?;

p.checkMove(2, 1);
```



Polymorphisme

Ce concept est fondamental en programmation orientée objet. Pour avoir des codes modulaire et flexible, il est recommandé de typer ses variables sur des abstractions comme des interfaces ou des classes concrète

Cela permet d’ajouter facilement de nouveaux comportement à un programme : Il suffit d’ajouter une nouvelle classe à une famille de classe



Polymorphisme

Se baser sur des abstractions permet de se concentrer sur l’algorithme plutôt que l’implémentation.

Le code devient flexible car il est possible d’interchanger les classes utilisées sans changer l’algorithme

```
while(winner == false) {
    Scanner in = new Scanner(System.in);
    int x = in.nextInt();
    int y = in.nextInt();

    // le vrai type de p ne nous interesse pas et ne change pas l'algo
    p.checkMove(x,y);

    ...
}
```

Polymorphisme

program to interfaces not implementations

Design Patterns: Elements of Reusable Object-Oriented Software - Gang of Four (GoF) 1995

Exemple de polymorphisme

Redéfinir la méthode toString d’une classe

Exercices